

Operaciones Básicas con Matrices 2x2 en Python

Documento de Estudio

20 de febrero de 2026

Índice

1. Introducción	2
2. Representación de Matrices en Python	2
3. Operaciones Básicas	2
3.1. Suma y Resta de Matrices	2
3.2. Multiplicación por Escalar	3
3.3. Multiplicación de Matrices	3
3.4. Determinante	4
3.5. Matriz Transpuesta	4
3.6. Matriz Inversa	5
4. Programa Completo	5
5. Ejercicios Prácticos	7
6. Conclusión	7

1. Introducción

Las matrices 2x2 son fundamentales en álgebra lineal y tienen numerosas aplicaciones en:

- Transformaciones geométricas
- Sistemas de ecuaciones lineales
- Gráficos por computadora
- Análisis de datos

En este documento, exploraremos las operaciones básicas implementadas en Python.

2. Representación de Matrices en Python

En Python, podemos representar matrices 2x2 usando listas anidadas:

```
1 # Representación de una matriz 2x2
2 matriz_a = [[1, 2],
3             [3, 4]]
4
5 matriz_b = [[5, 6],
6             [7, 8]]
7
8 print("Matriz A:")
9 for fila in matriz_a:
10     print(fila)
```

Listing 1: Representación de matrices 2x2

3. Operaciones Básicas

3.1. Suma y Resta de Matrices

La suma y resta de matrices se realiza elemento a elemento:

```
1 def suma_matrices(A, B):
2     """Suma dos matrices 2x2"""
3     resultado = [[0, 0],
4                  [0, 0]]
5     for i in range(2):
6         for j in range(2):
7             resultado[i][j] = A[i][j] + B[i][j]
8     return resultado
9
10 def resta_matrices(A, B):
11     """Resta dos matrices 2x2"""
12     resultado = [[0, 0],
13                  [0, 0]]
14     for i in range(2):
```

```

15         for j in range(2):
16             resultado[i][j] = A[i][j] - B[i][j]
17         return resultado
18
19 # Ejemplo
20 A = [[1, 2], [3, 4]]
21 B = [[5, 6], [7, 8]]
22
23 suma = suma_matrices(A, B)
24 print("Suma:", suma)
25
26 resta = resta_matrices(A, B)
27 print("Resta:", resta)

```

Listing 2: Suma y resta de matrices

3.2. Multiplicación por Escalar

Multiplicamos cada elemento por un número escalar:

```

1 def multiplicacion_escalar(matriz, escalar):
2     """Multiplica una matriz por un escalar"""
3     resultado = [[0, 0],
4                  [0, 0]]
5     for i in range(2):
6         for j in range(2):
7             resultado[i][j] = matriz[i][j] * escalar
8     return resultado
9
10 # Ejemplo
11 A = [[1, 2], [3, 4]]
12 escalar = 3
13 resultado = multiplicacion_escalar(A, escalar)
14 print(f"Matriz {escalar}:", resultado)

```

Listing 3: Multiplicación por escalar

3.3. Multiplicación de Matrices

La multiplicación de matrices 2x2 sigue la regla:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \cdot \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}$$

```

1 def multiplicacion_matrices(A, B):
2     """Multiplica dos matrices 2x2"""
3     resultado = [[0, 0],
4                  [0, 0]]
5
6     resultado[0][0] = A[0][0] * B[0][0] + A[0][1] * B[1][0]
7     resultado[0][1] = A[0][0] * B[0][1] + A[0][1] * B[1][1]
8     resultado[1][0] = A[1][0] * B[0][0] + A[1][1] * B[1][0]

```

```

9      resultado[1][1] = A[1][0] * B[0][1] + A[1][1] * B[1][1]
10
11     return resultado
12
13 # Ejemplo
14 A = [[1, 2], [3, 4]]
15 B = [[5, 6], [7, 8]]
16 producto = multiplicacion_matrices(A, B)
17 print("Producto:", producto)

```

Listing 4: Multiplicación de matrices

3.4. Determinante

El determinante de una matriz 2x2 se calcula como:

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

```

1 def determinante(matriz):
2     """Calcula el determinante de una matriz 2x2"""
3     return matriz[0][0] * matriz[1][1] - matriz[0][1] * matriz
4         [1][0]
5
6 # Ejemplo
7 A = [[1, 2], [3, 4]]
8 det = determinante(A)
9 print(f"Determinante de {A}: {det}")

```

Listing 5: Cálculo del determinante

3.5. Matriz Transpuesta

La matriz transpuesta intercambia filas por columnas:

```

1 def transpuesta(matriz):
2     """Calcula la transpuesta de una matriz 2x2"""
3     return [[matriz[0][0], matriz[1][0]],
4             [matriz[0][1], matriz[1][1]]]
5
6 # Ejemplo
7 A = [[1, 2], [3, 4]]
8 trans = transpuesta(A)
9 print("Matriz original:", A)
10 print("Transpuesta:", trans)

```

Listing 6: Matriz transpuesta

3.6. Matriz Inversa

La inversa de una matriz 2x2 existe si su determinante es diferente de cero:

$$A^{-1} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

```
1 def inversa(matriz):
2     """Calcula la inversa de una matriz 2x2"""
3     det = determinante(matriz)
4
5     if det == 0:
6         return "La matriz no tiene inversa (determinante = 0)"
7
8     inv_det = 1 / det
9     return [[matriz[1][1] * inv_det, -matriz[0][1] * inv_det],
10             [-matriz[1][0] * inv_det, matriz[0][0] * inv_det]]
11
12 # Ejemplo
13 A = [[4, 7], [2, 6]]
14 inv = inversa(A)
15 print("Matriz original:", A)
16 print("Inversa:", inv)
17
18 # Verificaci n: A * A^(-1) debe ser la matriz identidad
19 identidad = multiplicacion_matrices(A, inv)
20 print("Verificaci n (debe ser matriz identidad):", identidad)
```

Listing 7: Cálculo de la matriz inversa

4. Programa Completo

Aquí tienes un programa completo que incluye todas las operaciones:

```
1 class Matriz2x2:
2     """Clase para manejar matrices 2x2"""
3
4     def __init__(self, a11, a12, a21, a22):
5         self.matriz = [[a11, a12], [a21, a22]]
6
7     def __str__(self):
8         return f"[{self.matriz[0][0]:4} {self.matriz[0][1]:4}]\n"
9             f"[{self.matriz[1][0]:4} {self.matriz[1][1]:4}]"
10
11     def suma(self, otra):
12         """Suma dos matrices"""
13         return Matriz2x2(
14             self.matriz[0][0] + otra.matriz[0][0],
15             self.matriz[0][1] + otra.matriz[0][1],
16             self.matriz[1][0] + otra.matriz[1][0],
17             self.matriz[1][1] + otra.matriz[1][1])
```

```

18
19 def producto(self, otra):
20     """Multiplica dos matrices"""
21     return Matriz2x2(
22         self.matriz[0][0]*otra.matriz[0][0] + self.matriz
23         [0][1]*otra.matriz[1][0],
24         self.matriz[0][0]*otra.matriz[0][1] + self.matriz
25         [0][1]*otra.matriz[1][1],
26         self.matriz[1][0]*otra.matriz[0][0] + self.matriz
27         [1][1]*otra.matriz[1][0],
28         self.matriz[1][0]*otra.matriz[0][1] + self.matriz
29         [1][1]*otra.matriz[1][1]
30     )
31
32 def determinante(self):
33     """Calcula el determinante"""
34     return self.matriz[0][0]*self.matriz[1][1] - self.matriz
35         [0][1]*self.matriz[1][0]
36
37 def inversa(self):
38     """Calcula la inversa"""
39     det = self.determinante()
40     if det == 0:
41         return None
42     inv_det = 1/det
43     return Matriz2x2(
44         self.matriz[1][1]*inv_det,
45         -self.matriz[0][1]*inv_det,
46         -self.matriz[1][0]*inv_det,
47         self.matriz[0][0]*inv_det
48     )
49
50 # Ejemplo de uso
51 if __name__ == "__main__":
52     A = Matriz2x2(1, 2, 3, 4)
53     B = Matriz2x2(5, 6, 7, 8)
54
55     print("Matriz A:")
56     print(A)
57     print("\nMatriz B:")
58     print(B)
59
60     print("\nSuma A + B:")
61     print(A.suma(B))
62
63     print("\nProducto A * B:")
64     print(A.producto(B))
65
66     print(f"\nDeterminante de A: {A.determinante()}")
67
68     print("\nInversa de A:")

```

```

64     inv_A = A.inversa()
65     if inv_A:
66         print(inv_A)
67         print("\nVerificaci n A * A^(-1):")
68         print(A.producto(inv_A))
69     else:
70         print("La matriz A no tiene inversa")

```

Listing 8: Programa completo de operaciones con matrices

5. Ejercicios Prácticos

1. Calcula la suma de las matrices $\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$ y $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
2. Encuentra el producto de $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ por sí misma
3. Determina si la matriz $\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$ tiene inversa
4. Verifica que $(AB)^T = B^T A^T$ para matrices 2x2

6. Conclusión

Las operaciones básicas con matrices 2x2 son fundamentales y su implementación en Python nos permite:

- Visualizar mejor los conceptos matemáticos
- Realizar cálculos complejos de manera eficiente
- Verificar propiedades y teoremas del álgebra lineal
- Aplicar estos conceptos a problemas prácticos

La comprensión de estas operaciones es esencial para avanzar en temas más complejos como espacios vectoriales, transformaciones lineales y autovalores.